

Paso 26A — preparar TRAIN / VALIDATION / TEST

A partir de aquí recomiendo PySpark ML.

```
from pyspark.sql import functions as F

model_df = (
    spark.table("bu_frd.model_features_ab_final_2022_2025")
    .withColumn("label", F.col("label_3m").cast("double"))
)

train_df = model_df.filter(F.col("model_split") == "TRAIN")
validation_df = model_df.filter(F.col("model_split") == "VALIDATION")
test_df = model_df.filter(F.col("model_split") == "TEST")

print("TRAIN:", train_df.count())
print("VALIDATION:", validation_df.count())
print("TEST:", test_df.count())
```

Paso 26B — features de Model A

```
features_A = [

    # Sparse behavior
    "deterioration_acceleration",
    "adverse_acceleration",
    "max_dpd_jump_recent_3m",

    # Active account behavior
    "active_accounts_change_vs_baseline",
    "active_accounts_changed_flag",

    # Balance
    "balance_change_model",
    "log1p_abs_balance_change_model",
    "balance_change_available_flag",
    "balance_recent_months_available",
    "balance_prior_months_available",
```

```

# Past due
"past_due_change_model",
"past_due_change_available_flag",
"past_due_recent_months_available",
"past_due_prior_months_available",

# Delinquency
"delinquency_share_change_model",
"delinquency_change_available_flag",
"delinquency_recent_months_available",
"delinquency_prior_months_available",

# Utilization
"avg_utilization_change_model",
"weighted_utilization_change_model",
"utilization_change_available_flag",
"utilization_recent_months_available",
"utilization_prior_months_available"
]

```

Paso 26C — Model B

```

features_B = features_A + [

    "active_accounts_z_model",
    "active_accounts_z_available_flag",

    "avg_utilization_z_model",
    "avg_utilization_z_available_flag",

    "weighted_utilization_z_model",
    "weighted_utilization_z_available_flag"
]

```

Paso 26D — assembler

```

from pyspark.ml.feature import VectorAssembler

assembler_A = VectorAssembler(
    inputCols=features_A,
    outputCol="features",
    handleInvalid="error"
)

assembler_B = VectorAssembler(

```

```

        inputCols=features_B,
        outputCol="features",
        handleInvalid="error"
    )

```

Paso 26G — función para entrenar A/B

```

from pyspark.ml.classification import LogisticRegression

```

```

def fit_logistic(
    train,
    validation,
    assembler,
    reg_param,
    model_name
):

```

```

    train_vec = assembler.transform(train)
    validation_vec = assembler.transform(validation)

```

```

    lr = LogisticRegression(
        featuresCol="features",
        labelCol="label",
        predictionCol="prediction",
        probabilityCol="probability",
        rawPredictionCol="rawPrediction",

```

```

        regParam=reg_param,
        elasticNetParam=0.0,

```

```

        standardization=True,
        fitIntercept=True,
        maxIter=100
    )

```

```

    model = lr.fit(train_vec)

```

```

    predictions = model.transform(validation_vec)

```

```

    return model, predictions

```

Paso 26H — métricas principales

```

from pyspark.ml.evaluation import BinaryClassificationEvaluator

```

```

pr_evaluator = BinaryClassificationEvaluator(
    labelCol="label",

```

```

        rawPredictionCol="rawPrediction",
        metricName="areaUnderPR"
    )

roc_evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

```

Paso 26I — obtener score

```

from pyspark.ml.functions import vector_to_array

def add_score(predictions):

    return predictions.withColumn(
        "score",
        vector_to_array("probability")[1]
    )

```

Paso 26J — Lift / Precision / Recall

```

def top_metrics(predictions, top_pct):

    scored = add_score(predictions)

    q = 1.0 - top_pct

    threshold = scored.approxQuantile(
        "score",
        [q],
        0.0001
    )[0]

    total_stats = scored.agg(
        F.count("*").alias("n"),
        F.sum("label").alias("cases"),
        F.avg("label").alias("base_rate")
    ).first()

    selected = scored.filter(
        F.col("score") >= threshold
    )

    top_stats = selected.agg(

```

```

        F.count("").alias("selected_n"),
        F.sum("label").alias("selected_cases"),
        F.avg("label").alias("positive_rate")
    ).first()

    precision = top_stats["positive_rate"]

    recall = (
        top_stats["selected_cases"]
        / total_stats["cases"]
    )

    lift = (
        top_stats["positive_rate"]
        / total_stats["base_rate"]
    )

    selected_share = (
        top_stats["selected_n"]
        / total_stats["n"]
    )

    return {
        "requested_top_pct": top_pct,
        "actual_selected_pct": selected_share,
        "threshold": threshold,
        "precision": precision,
        "recall": recall,
        "lift": lift
    }

```

Paso 26K — función completa de evaluación

```

def evaluate_model(predictions):

    auprc = pr_evaluator.evaluate(predictions)
    auroc = roc_evaluator.evaluate(predictions)

    top1 = top_metrics(predictions, 0.01)
    top5 = top_metrics(predictions, 0.05)
    top10 = top_metrics(predictions, 0.10)

    return {
        "AUPRC": auprc,
        "AUROC": auroc,

        "Lift_1": top1["lift"],

```

```

"Precision_1": top1["precision"],
"Recall_1": top1["recall"],

"Lift_5": top5["lift"],
"Precision_5": top5["precision"],
"Recall_5": top5["recall"],

"Lift_10": top10["lift"],
"Precision_10": top10["precision"],
"Recall_10": top10["recall"]
}

```

Paso 26L — correr A y B

```
results = []
```

```
for reg in reg_params:
```

```

# =====
# MODEL A
# =====

```

```

model_A, pred_A = fit_logistic(
    train_df,
    validation_df,
    assembler_A,
    reg,
    "MODEL_A"
)

```

```
metrics_A = evaluate_model(pred_A)
```

```

metrics_A["model"] = "A_CURRENT"
metrics_A["regParam"] = reg

```

```
results.append(metrics_A)
```

```

# =====
# MODEL B
# =====

```

```

model_B, pred_B = fit_logistic(
    train_df,
    validation_df,
    assembler_B,
    reg,

```

```
    "MODEL_B"  
  )
```

```
metrics_B = evaluate_model(pred_B)
```

```
metrics_B["model"] = "B_CURRENT_PLUS_Z"  
metrics_B["regParam"] = reg
```

```
results.append(metrics_B)
```

Luego:

```
results_df = spark.createDataFrame(results)
```

```
results_df.orderBy(  
    F.desc("AUPRC")  
)<show(  
    truncate=False  
)
```